

Count

计数 DP

计数 DP 是一种利用类似 DP 的记忆化搜索方法（与在狭义上的 DP，即最优化问题有一定区别），用于解决计数（以及求和）问题。

基础

基本思想

计数问题一般指求一个集合 的大小，在 OI 中， 的大小有时会达到 甚至 的级别（当然，一般会对某一个固定的数取模），其中 是问题规模，所以我们不能逐一求出 的元素。

如果我们能够将 分成若干无交的子集，那么 的元素个数就等于这些部分的元素个数的和。如果这些子集的计数恰好与原问题类似，那么我们就可以通过类似动态规划的方法来解决。

例题

▼ 例题

给定一个正整数 n ，求有多少个把 n 划分成 k 个正整数的和的方案，位置调换视为不同的划分方案。

需集合 S 为形如 $\{x_1, x_2, \dots, x_k\}$ 的正整数数组组成的集合，其中 $x_i \geq 1$ 。如果 k 固定，则有如下推导：因为 $x_i \geq 1$ ，所以 $x_i = 1 + y_i$ 。根据 y_i 的定义， $y_i \geq 0$ 。

由于 n 是正整数，所以 y_i 的取值范围是 $0 \leq y_i \leq n/k - 1$ 。因此， S 可以按照 y_i 被划分，分成 $(n/k + 1)^k$ 个子集，其中当 $y_i = 0$ 时，这个子集为：

这个子集的元素个数显然等于 1 ，由于 y_i 的不同，这些子集两两无交。所以：

这样我们就可以使用类似 DP 的方法处理它：设 $f(n, k)$ 为 n 划分成 k 个正整数的和的方案数，则有状态转移方程：

这样就可以使用 DP 的方法求解了。

与最优化 DP 的异同

可以发现，计数 DP 和最优化 DP 都是在一个范围 $[0, n]$ 内求一个值（大小值、最优值），这个值通过将 S 中的所有元素做一次处理，再对处理值做一次整合得到。

例如，对于 0-1 背包问题， S 中的元素为背包内的所有物品组成的集合，对于 S 中的一个方案，我们对 S 做一次处理，处理得到的结果 val 为 S 中物品的总价值，对所有得到的处理值，我们取最大值，得到问题的答案。

对于计数问题， S 中的元素为要计算元素个数的集合，它的处理是把所有的 S 中元素变为 1 ，然后将这些 1 通过加法的方式汇总起来，因为每一个 S 中元素都对应一个 1 ，所以这样得到的值就是 S 中元素个数。

当汇总操作为最大/最小值时，我们可以将 S 分成任意若干个部分，只需这些部分的并为 S 即可，无需无交的条件。而计数问题由于不满足这个条件，所以我们需要将 S 分成若干个部分，这些部分两两无交，这就是与最优化 DP 的区别。

例题

▼ 例题

给定一个正整数 n ，求有多少个把 n 划分成任意多个正整数的和的方案，位置调换视为 **相同** 的划分方案。

解法 1

需要计算的集合的元素为满足其和为 n 的正整数多重集。但是这样显然不好推。

若一个多重集 S 只包含 n 的正整数，且 S 中所有元素的和为 n ，则称 S 为 n 的划分。考虑 S 出现的个数。可能为 1 。于是它可以被转移到 $n-1$ 。求和一下即可。复杂度是 $O(n^2)$ （来自于 n 的范围导致的调和级数）。

但是这样还不够优秀。考虑下面所示的一个例子：

等量代换得 $f(n) = f(n-1) + f(n-2)$ 。同理我们可以得到一个通用的状态转移方程：

此时，时间复杂度为 $O(n^2)$ 。

解法 2

考虑到某一个正整数组成的多重集 S 必然可以通过「将 S 中每一个元素自增」、「在 S 中加一个值为 1 的元素」两个操作得到，并且不同的操作序列得到的结果是不同的。

这样对 S 的转移可以变为对操作序列的转移。考虑将 S 划分成 k 个数的操作序列（所有的这些操作序列记作 σ ）中的最后一次操作，如果是 $+$ 操作，那么不会增加数，但是 k 增加了 1 。为了使最终的 k ，原来的 σ （记作 σ' ）的和需要为 $n-1$ 。所以；如果是 1 操作，那么会增加一个数， k 增加了 1 。所以 k 。

这样做的时间复杂度依旧是 $O(n^2)$ 。

解法 3

考虑将 n 分为大于 1 的部分和小于等于 1 的部分。可以使用解法 1 求出 $f(n-1)$ ，而 $f(n)$ 的数量可以通过略微修改解法 2 求出：考虑将两个操作变为「将 S 中每一个元素自增」、「在 S 中加一个值为 1 的元素」。容易列出状态转移方程。

将 n 拆为 k 和 $n-k$ 两部分。枚举其中一个即可得出另一个。将满足 k 的个数和 $n-k$ 的个数求出，乘起来，对所有的 k 求和便是最终结果。

由于在计算 k 个数的过程中， $f(k)$ ，所以我们利用解法 1 计算 $f(k)$ 的时间复杂度为 $O(k^2)$ 。同样地，由于在计算 $n-k$ 个数的过程中， $f(n-k)$ ，所以我们利用解法 2 计算 $f(n-k)$ 的时间复杂度也是 $O(k^2)$ 。所以总时间复杂度为 $O(n^3)$ 。

本页面最近更新：2024/1/13 02:05:49, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [FFjet](#), [lr1d](#), [Oler1048576](#), [Tiphereth-A](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用